

2. Local Agent 구성 정리

2. Local Agent

2-1. 이 프로세스의 정체

Local Agent는 **Host Simulator**의 상태를 읽어서, 그것을 네트워크로 보낼 이벤트 메시지로 바꾸는 프로세스다. 즉 이 프로세스는 다음 두 세계를 연결한다.

- 로컬 상태 세계

Host Simulator의 cpu/memory/service 상태

- 네트워크 메시지 세계

EVENT 메시지

그래서 이 프로세스가 실질적으로 **로컬 계층과 네트워크 계층의 경계**다.

2-2. 왜 필요한가

Host Simulator는 그냥 상태만 만든다.

그 상태를 그대로 Relay에 보낼 수는 없다.

예를 들어 Host Simulator는 이렇게만 가지고 있다.

```
cpu=95
service=DOWN
latency=HIGH
```

이걸 Relay가 이해할 수 있는 이벤트로 바꿔야 한다.

예:

```
{
  "event_id": "evt-1042",
  "host_id": "host-1",
  "type": "CPU_SPIKE",
  "severity": "WARN",
  "timestamp": "...",
  "seq_no": 41,
  "payload": {
    "cpu": 95
  }
}
```

이 변환을 담당하는 것이 Local Agent다.

2-3. Local Agent의 목적

목적은 크게 4개다.

1. Host 상태 읽기

- Host Simulator가 현재 어떤 상태인지 확인

2. 상태 변화 또는 임계치 초과 감지

- 예: CPU가 80% 넘었는가
- 서비스가 UP에서 DOWN으로 바뀌었는가

3. 이벤트 생성

- 감지된 상태를 EVENT 메시지로 변환

4. 네트워크로 publish

- Relay R1에 메시지 전송 시작

즉 이 프로세스는 단순 reader가 아니라

상태 판단 + 이벤트 생성 + 송신 시작점이다.

2-4. 입력 / 출력

입력

Local Agent의 입력은 크게 3종류다.

A. Host Simulator 상태

주기적으로 읽는다.

예:

```
{
  "host_id": "host-1",
  "cpu": 95,
  "memory": 48,
  "disk": 62,
  "service": "UP",
  "latency": "NORMAL",
  "alive": true
}
```

B. Controller의 제어 명령

예:

- start
- pause
- reset
- set threshold
- set publish interval

C. 내부 시간 정보

- polling 주기
- debounce 시간
- 재전송 여부 판단용 로컬 시간

출력

Local Agent의 출력은 보통 아래와.

A. EVENT 메시지

Relay R1로 전송

B. 상태 로그

예:

- "CPU threshold exceeded"
- "Service down detected"
- "Event published to R1"

C. 내부 상태 갱신

예:

- last_seen_status 갱신
- seq_no 증가
- 최근 보낸 이벤트 기록

2-5. Local Agent 내부에 있어야 하는 상태값

실제 구현 기준으로 최소한 아래는 있어야 한다.

```
agent_id
host_id
running_state
poll_interval
cpu_threshold
memory_threshold
last_host_status
last_sent_seq_no
last_event_time_by_type
dedicated_event_counter
r1_address
publish_enabled
```

각 의미는 이렇다.

agent_id

- Agent 프로세스 식별자
- 예: `agent-host-1`

host_id

- 이 Agent가 담당하는 host 식별자

running_state

- `RUNNING`, `PAUSED`, `STOPPED`

poll_interval

- 상태 읽기 주기
- 예: 1초

cpu_threshold

- CPU 이벤트 발생 기준
- 예: 80

memory_threshold

- 메모리 이벤트 발생 기준
- 예: 85

last_host_status

- 직전 polling에서 읽은 상태
- 변화 감지를 위해 필요

last_sent_seq_no

- 이벤트 순서 번호
- 생성할 때마다 증가

last_event_time_by_type

- 같은 이벤트를 너무 자주 보내지 않기 위한 기록
- 예: CPU_SPIKE를 1초마다 무한 발행하지 않게 함

r1_address

- 메시지를 보낼 R1의 주소

publish_enabled

- 현재 외부 publish 허용 여부

2-6. Local Agent가 담당하는 핵심 기능

이제 기능을 하나씩 본다.

기능 1. Host 상태 polling

목적

Host Simulator의 현재 상태를 주기적으로 읽는다.

작동 방식

매 `poll_interval` 마다 `get_status()` 또는 로컬 인터페이스 호출

실제 작동

1초마다:

- Host Simulator 상태 조회
- 현재 cpu/memory/service/latency 값 받음

중요한 이유

이걸 해야 상태 변화가 발생했는지 알 수 있다.

기능 2. 상태 비교 및 변화 감지

목적

현재 상태와 이전 상태를 비교해서 이벤트가 필요한지 판단

작동 방식

현재 상태와 `last_host_status` 를 비교한다.

예:

- CPU: 32 → 95
- Service: UP → DOWN
- Latency: NORMAL → HIGH

실제 작동

비교 방식은 두 종류다.

A. 임계치 기반

- CPU > 80 이면 이벤트 발생

B. 상태 전이 기반

- Service가 UP에서 DOWN으로 바뀌면 이벤트 발생

이 두 방식이 함께 필요하다.

기능 3. 이벤트 발생 조건 판단

목적

모든 상태 변화를 다 보내지 않고, 의미 있는 것만 이벤트로 만든다.

작동 방식

다음처럼 판단 규칙을 둔다.

예:

- `cpu >= 80` → `CPU_SPIKE`
- `memory >= 85` → `MEMORY_PRESSURE`
- `service == DOWN` → `SERVICE_DOWN`
- `latency == HIGH` → `LATENCY_HIGH`
- `service가 DOWN → UP` → `SERVICE_RECOVERED`

실제 작동

Host 상태는 계속 변하지만, 이벤트는 조건을 만족할 때만 발생한다.

기능 4. 이벤트 메시지 생성

목적

감지된 상태를 네트워크용 메시지 형식으로 바꾼다.

작동 방식

이벤트마다 아래 정보를 채운다.

- `event_id`
- `host_id`
- `agent_id`
- `event_type`
- `severity`
- `timestamp`
- `seq_no`
- `payload`

실제 예시

```
{
  "event_id": "evt-00041",
  "host_id": "host-1",
  "agent_id": "agent-host-1",
  "type": "SERVICE_DOWN",
  "severity": "CRITICAL",
```

```
"timestamp": "2026-03-19T10:00:13",
"seq_no": 41,
"payload": {
  "service": "DOWN"
}
}
```

기능 5. 중복 발행 억제

목적

같은 상태를 너무 자주 연속 발행하는 것을 막는다.

예를 들어 CPU가 95 상태로 10초 동안 유지되면,

매 1초마다 **CPU_SPIKE** 를 계속 보내는 건 비효율적일 수 있다.

작동 방식

같은 타입 이벤트에 대해 **cooldown** 또는 **state transition 기반 발행**을 한다.

방법은 두 가지다.

A. 상태 전이 시에만 발행

- 정상 → 비정상으로 바뀔 때 한 번
- 비정상 → 정상으로 복구될 때 한 번

B. 주기적 재통보

- 같은 장애가 계속되면 10초마다 재알림

초기 구현은 **A 방식**이 가장 단순하다.

기능 6. Relay R1로 publish

목적

생성한 이벤트를 네트워크 시작점으로 보낸다.

작동 방식

이벤트 메시지를 R1 주소로 전송한다.

주의:

- Local Agent는 보통 **전달 보장 전체를 담당하지는 않는다**
- 하지만 최소한 **송신 시작점**으로서 publish는 해야 한다

설계에 따라:

- Agent도 ACK를 직접 기다릴 수 있고
- 또는 R1에 넘기기만 할 수도 있다

초기 설계에서는 Agent는 **R1에 전송하는 역할 중심**으로 두는 것이 단순하다.

기능 7. 실행 제어

목적

실험을 위해 Agent를 제어 가능하게 한다.

지원해야 할 명령

- start
- pause
- resume
- reset
- threshold 변경

실제 작동

- `pause` 상태에서는 polling만 멈추거나 publish를 멈춤
- `reset` 상태에서는 내부 seq나 last state를 초기화

2-7. Agent가 감지해야 하는 이벤트 종류

최소 기준으로 아래는 있어야 한다.

A. CPU_SPIKE

발생 조건

- `cpu_usage >= cpu_threshold`

목적

CPU 과부하 탐지

B. MEMORY_PRESSURE

발생 조건

- `memory_usage >= memory_threshold`

목적

메모리 과부하 탐지

C. SERVICE_DOWN

발생 조건

- `service_state == DOWN`
- 직전 상태는 `UP`

목적

서비스 중단 탐지

D. LATENCY_HIGH

발생 조건

- `latency_state == HIGH`

목적

네트워크 상태 악화 탐지

E. RECOVERY 이벤트

예:

- `CPU_RECOVERED`
- `SERVICE_RECOVERED`
- `LATENCY_NORMALIZED`

발생 조건

- 비정상 상태에서 정상 상태로 되돌아옴

목적

상태 복구를 명확히 알리기 위함

2-8. Local Agent의 실제 작동 흐름

이제 이 프로세스만 떼어놓고 흐름으로 보자.

정상 상태 흐름

1. Agent 시작
2. `poll_interval`마다 Host 상태 조회
3. 직전 상태와 비교
4. 이벤트 조건 미충족
5. `last_host_status`만 갱신
6. 다음 주기 대기

즉 정상 상태에서는 보통 이벤트를 거의 만들지 않는다.

CPU Spike 감지 흐름

1. Agent가 Host 상태 조회
2. `cpu=95` 확인
3. 직전 상태는 `cpu=22`

4. threshold 초과 판단
5. CPU_SPIKE 이벤트 생성
6. event_id / seq_no 부여
7. R1로 publish
8. last_host_status 갱신

Service Down 감지 흐름

1. Host 상태 조회
2. service=DOWN 확인
3. 직전 상태는 service=UP
4. 상태 전이 감지
5. SERVICE_DOWN 이벤트 생성
6. R1로 publish
7. 내부 로그 기록

Recovery 감지 흐름

1. Host 상태 조회
2. service=UP 확인
3. 직전 상태는 DOWN
4. recovery 판단
5. SERVICE_RECOVERED 이벤트 생성
6. publish

2-9. Local Agent가 사용하는 판정 로직

이 부분이 구현 핵심이다.

Agent는 단순히 “값 읽기”만 하는 게 아니라

“이 값이 이벤트로 바뀌어야 하는가?”를 결정해야 한다.

예를 들면 다음 순서다.

- 현재 상태 읽기
 - 이전 상태와 비교
 - 임계치 초과 여부 판단
 - 상태 전이 여부 판단
 - 이미 같은 이벤트를 방금 보냈는지 확인
 - 필요하면 이벤트 생성
 - publish

즉 Agent는 일종의 로컬 이벤트 판단기다.

2-10. event_id와 seq_no는 왜 필요한가

event_id

- 이벤트 자체를 유일하게 식별
- Relay와 Monitor가 dedup할 때 사용

예:

- `evt-host1-000041`

seq_no

- host 단위 이벤트 순서를 표현
- ordering 문제 추적에 사용

예:

- Host에서 41번째 이벤트

둘은 역할이 다르다.

- `event_id` : 중복 제거용
- `seq_no` : 순서 확인용

2-11. 권장 이벤트 메시지 구조

Agent가 만들어야 할 메시지는 최소한 아래 필드가 있어야 한다.

```
{
  "event_id": "evt-host1-000041",
  "seq_no": 41,
  "host_id": "host-1",
  "agent_id": "agent-host-1",
  "type": "CPU_SPIKE",
  "severity": "WARN",
  "timestamp": "2026-03-19T10:00:13",
  "payload": {
    "cpu": 95
  }
}
```

각 필드 목적:

- `event_id` : dedup
- `seq_no` : ordering 추적
- `host_id` : 어느 host인지
- `agent_id` : 누가 만들었는지
- `type` : 이벤트 종류

- **severity** : 경고 수준
- **timestamp** : 발생 시각
- **payload** : 상세 값

2-12. 이 프로세스가 하지 않는 것

헛갈리면 안 되는 부분이다. Local Agent는 아래를 하지 않는다.

- R1/R2의 중간 relay 역할 안 함
- ACK/timeout 전체 라우팅 관리 안 함
- failover routing 결정 안 함
- 최종 로그 저장 안 함
- 대시보드 표시 안 함

즉 Local Agent는 **전달 시작점**이지

전체 전달 네트워크 관리자 아니다.

2-13. Local Agent가 꼭 있어야 하는 이유

이 프로세스가 없으면:

- Host 상태는 있지만
- 네트워크에 보낼 이벤트가 없다

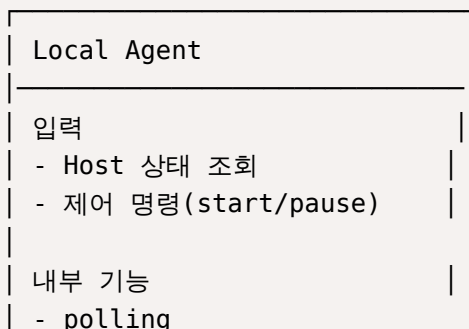
즉 Host Simulator만으로는 프로젝트가 시작되지 않는다.

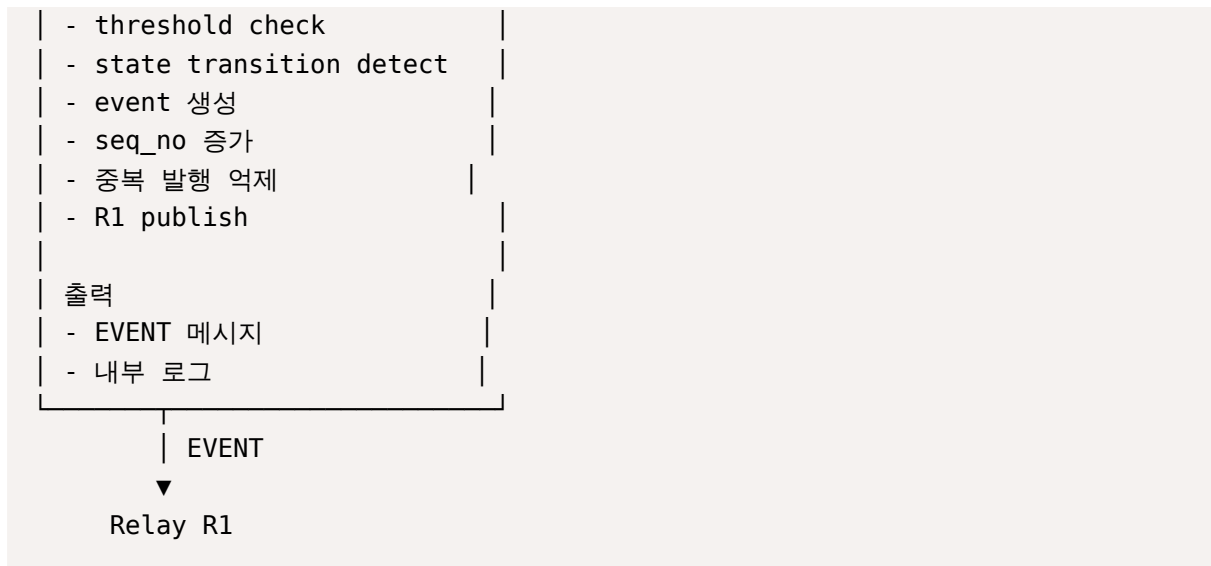
Local Agent가 있어야:

- 상태를 읽고
- 의미를 해석하고
- 이벤트로 만들고
- Relay 네트워크에 넘긴다

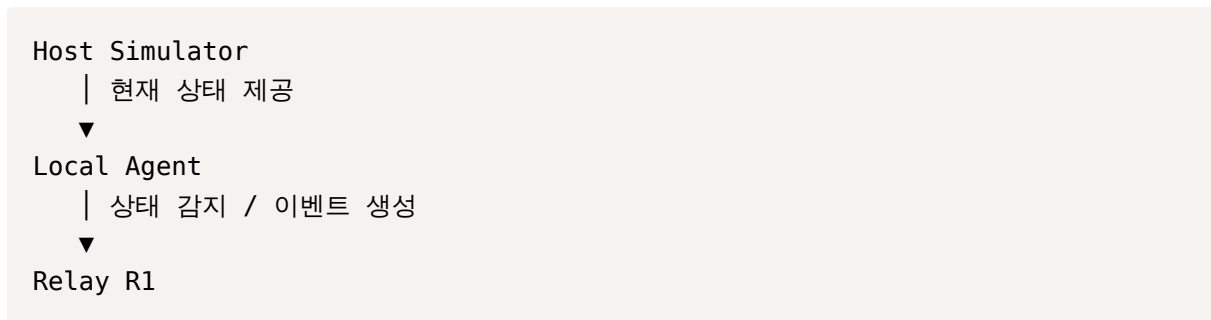
그래서 이 프로세스는 전체 시스템의 **실질적 출발점**이다.

2-14. Local Agent만 따로 구조도로 그리면





2-15. Host Simulator와 연결해서 보면

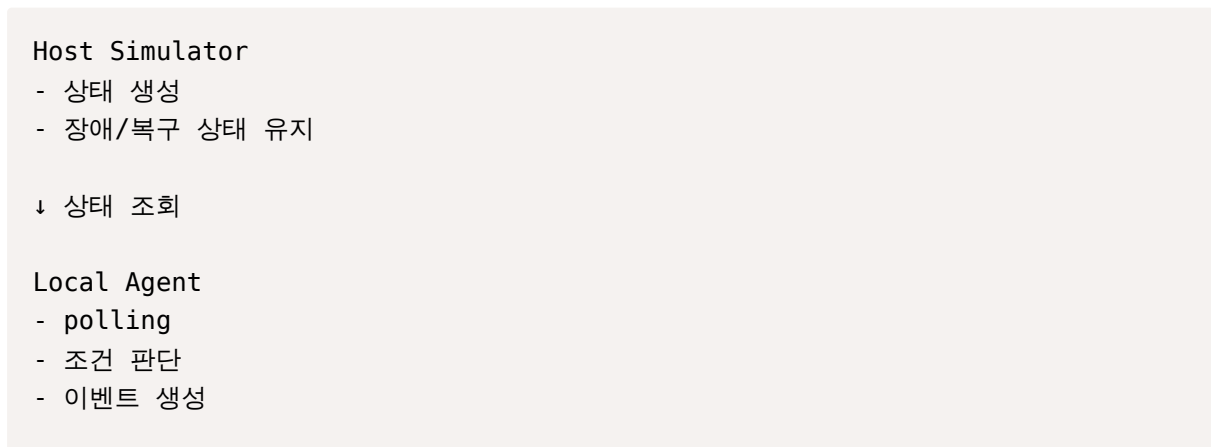


여기서부터 네트워크 계층이 시작된다.

2-16. 핵심 한 문장 정리

Local Agent는 Host Simulator의 상태를 주기적으로 읽고, 임계치 초과나 상태 전이를 감지하여 이벤트 메시지를 생성한 뒤 Relay R1으로 publish하는 프로세스다.

지금까지 Host + Agent 흐름 한 번에 정리



- event_id / seq_no 부여
- R1로 publish